G U I D E N O V A T E L

BigQuery Sandbox

Préparation et fiabilisation des données

Parcours Data Analyst · Octobre 2026

Chaîne cible

Fichiers CSV locaux → BigQuery Sandbox → contrôle qualité → nettoyage par vues → données prêtes pour Looker Studio

ℹ️ Important

Aucune carte bancaire n'est nécessaire pour ce brief. Nous utilisons BigQuery Sandbox, qui fonctionne sans compte de facturation. Le guide évite volontairement Cloud Storage et les opérations DML (UPDATE, DELETE, MERGE).

1. Objectif du guide

Vous allez préparer les données commerciales de NovaTel afin de produire une source fiable, structurée et réutilisable dans le prochain brief consacré à Looker Studio.

- charger des fichiers CSV locaux dans BigQuery Sandbox ;
- comprendre le modèle en étoile NovaTel ;
- contrôler la qualité des données ;
- interpréter les anomalies avant de les corriger ;
- appliquer les règles métier au moyen de requêtes SQL simples ;
- conserver les données brutes et créer des vues propres pour le reporting.

📌 Méthode pédagogique

Pour chaque anomalie : 1) détecter 2) observer 3) réfléchir 4) appliquer la correction fournie 5) vérifier le résultat.

Les requêtes sont un moyen ; l'objectif principal est de comprendre les données et leur impact métier.

2. BigQuery Sandbox : ce qu'il faut savoir

BigQuery Sandbox permet d'utiliser BigQuery sans carte bancaire et sans compte de facturation. Pour ce brief, ses limites sont compatibles avec nos besoins.

- jusqu'à 10 Gio de stockage dans le sandbox ;
- jusqu'à 1 Tio de données interrogées par mois ;
- les tables, vues et partitions du sandbox expirent après 60 jours ;
- les opérations DML telles que UPDATE, DELETE et MERGE ne sont pas disponibles ;

- les requêtes SELECT, les jointures, agrégations, CASE WHEN et les vues sont adaptées à ce projet.

✅ Conséquence pour NovaTel

Nous ne modifierons jamais la table brute. Les corrections seront intégrées dans des vues propres.

Cette approche conserve une source d'origine intacte et rend les règles de nettoyage visibles et reproductibles.

3. Accéder à BigQuery Sandbox

BigQuery Sandbox s'active en quelques minutes depuis n'importe quel compte Google. Suivez les étapes ci-dessous dans l'ordre.

1. **Ouvrir la console BigQuery**
   Rendez-vous sur : https://console.cloud.google.com/bigquery
2. **Se connecter avec un compte Google**
   Utilisez un compte Google existant (Gmail ou Google Workspace). Si vous n'en avez pas, créez-en un gratuitement sur accounts.google.com. Aucun compte de facturation n'est nécessaire.
3. **Créer un projet Google Cloud**
   À la première connexion, Google Cloud vous propose de créer un projet. Cliquez sur "Créer un projet", donnez-lui un nom (par exemple : novatell-formation), et confirmez. Ne créez pas de compte de facturation — ignorez ou fermez toute fenêtre qui vous le propose.
4. **Vérifier que le Sandbox est actif**
   Une fois dans BigQuery, un bandeau bleu indique "Vous utilisez BigQuery en mode Sandbox". Si ce bandeau est absent, vérifiez qu'aucun compte de facturation n'a été lié au projet.
5. **Refuser toute activation de compte de facturation**
   Si Google Cloud vous propose d'activer la facturation ou de profiter de 300 $ de crédits gratuits, cliquez sur "Ignorer" ou fermez la fenêtre. Ces propositions déclenchent la création d'un compte payant et ne sont pas nécessaires pour ce brief.

ℹ️ À retenir

Le mode Sandbox est actif dès que vous créez un projet sans lier de compte de facturation.

Vous pouvez passer du Sandbox à un compte payant à tout moment, mais jamais l'inverse.

Pour ce brief, restez en mode Sandbox.

4. Créer le dataset novatel

Un dataset BigQuery est l'espace logique dans lequel seront stockées les tables NovaTel. Créez-le avant de charger les fichiers.

1. **Dans le volet Explorer, cliquez sur les trois points à côté de votre projet**
   Sélectionnez "Créer un ensemble de données" (Create dataset).
2. **Nommez le dataset**
   Saisissez exactement : novatel (en minuscules, sans espace).
3. **Choisir la localisation**

Sélectionnez US (multi-région). Cela harmonise la promotion et évite les erreurs de région lors des jointures.

4. **Confirmer**
   Cliquez sur "Créer un ensemble de données". Le dataset novatel apparaît dans le volet Explorer.

5. Charger les fichiers CSV directement depuis l'ordinateur

Dans BigQuery, sélectionnez le dataset novatel puis choisissez Create table. Répétez l'opération pour chacun des six fichiers.

Paramètre	Valeur à saisir
Create table from	Upload
Select file	Browse → sélectionner le fichier CSV
File format	CSV
Destination — dataset	novatel
Destination — table	nom du fichier sans .csv (ex: fact_transactions)
Schema	Auto detect, puis vérifier les types avant de créer

Chargez les six fichiers dans cet ordre :

- fact_transactions.csv
- dim_client.csv
- dim_produit.csv
- dim_point_vente.csv
- dim_date.csv
- dim_type_operation.csv

💬 Questions de réflexion

1. Quelle table contient le plus de lignes ?
2. Pourquoi fact_transactions est-elle beaucoup plus volumineuse que les dimensions ?
3. Quelle différence faites-vous entre une table de faits et une table de dimension ?
4. Pourquoi est-il utile de conserver une version brute des données ?

6. Contrôler les schémas après import

Pour chaque table, ouvrez l'onglet Schema puis Preview. Vérifiez les types détectés par BigQuery.

💬 Questions de réflexion

5. Quels types BigQuery retrouvez-vous principalement ?
6. Pourquoi le type d'une colonne est-il important ?
7. Que se passerait-il si un montant était importé comme STRING ?
8. Quelle table représente le centre du modèle en étoile ?

7. Première exploration SQL

Dans les requêtes suivantes, remplacez VOTRE_PROJET par l'identifiant de votre projet Google Cloud.

7.1 Compter les transactions

```
SELECT COUNT(*) AS nombre_transactions
FROM `VOTRE_PROJET.novatel.fact_transactions`;
```

💬 Questions de réflexion

9. Combien de transactions obtenez-vous ?
10. Le résultat correspond-il approximativement au volume annoncé ?
11. Pourquoi effectuer ce contrôle juste après un import ?

7.2 Observer quelques lignes

```
SELECT *
FROM `VOTRE_PROJET.novatel.fact_transactions`
LIMIT 20;
```

💬 Questions de réflexion

12. Repérez-vous déjà des valeurs inhabituelles ? Lesquelles ?

💡 Bonne pratique

Même dans le Sandbox, prenez l'habitude d'observer l'estimation du volume de données traité avant d'exécuter une requête.

8. Rechercher les dates manquantes

```
SELECT COUNT(*) AS dates_manquantes
FROM `VOTRE_PROJET.novatel.fact_transactions`
WHERE date_id IS NULL;
```

💬 Questions de réflexion

13. Combien de transactions n'ont pas de date ?
14. Pourquoi cela pose-t-il problème ?
15. Quelles analyses deviennent difficiles ou impossibles ?
16. Faut-il systématiquement supprimer une ligne dès qu'une valeur manque ?

✅ Décision retenue

Nous ne supprimons rien de la table brute. Les lignes sans date seront exclues uniquement de la vue destinée au reporting.

9. Détecter les doublons

```
SELECT
  transaction_id,
  COUNT(*) AS nombre_occurrences
FROM `VOTRE_PROJET.novatel.fact_transactions`
GROUP BY transaction_id
HAVING COUNT(*) > 1
ORDER BY nombre_occurrences DESC;
```

💬 Questions de réflexion

17. Combien d'identifiants apparaissent plusieurs fois ?
18. Pourquoi un doublon est-il particulièrement dangereux dans une table de transactions ?
19. Quels KPI seraient directement faussés ?
20. Quelle stratégie proposeriez-vous pour conserver une seule occurrence ?

⚠️ Correction

La déduplication sera intégrée plus loin dans la vue propre. La requête finale vous sera fournie afin de ne pas introduire de SQL avancé avant le cours dédié.

10. Contrôler les montants négatifs

```
SELECT
  transaction_id,
  montant_brut,
  remise,
  remboursement,
  montant_net,
  montant_encaisse
FROM `VOTRE_PROJET.novatel.fact_transactions`
WHERE montant_net < 0
ORDER BY montant_net;
```

💬 Questions de réflexion

21. Combien de montants nets négatifs trouvez-vous ?
22. Un montant négatif est-il forcément une erreur ?
23. Dans quels cas métier un montant négatif peut-il être normal ?

10.1 Ajouter le type d'opération

```
SELECT
  f.transaction_id,
  o.type_operation,
  f.montant_brut,
  f.remise,
  f.remboursement,
  f.montant_net
FROM `VOTRE_PROJET.novatel.fact_transactions` AS f
LEFT JOIN `VOTRE_PROJET.novatel.dim_type_operation` AS o
  ON f.type_operation_id = o.type_operation_id
WHERE f.montant_net < 0;
```

💬 Questions de réflexion

24. Les montants négatifs correspondent-ils tous à des remboursements ou des avoirs ?
25. Si ce n'est pas le cas, que peut-on en conclure ?
26. Quel serait l'impact sur le chiffre d'affaires ?

11. Comprendre CA brut, CA net et CA encaissé

```
SELECT
  SUM(montant_brut)     AS ca_brut,
  SUM(montant_net)      AS ca_net,
  SUM(montant_encaisse) AS ca_encaisse
FROM `VOTRE_PROJET.novatel.fact_transactions`;
```

💬 Questions de réflexion

27. Quelle différence observez-vous entre les trois indicateurs ?
28. Pourquoi le CA encaissé peut-il être inférieur au CA net ?
29. Une facture impayée contribue-t-elle au CA facturé ? au CA encaissé ?
30. Pourquoi cette distinction est-elle importante pour une direction financière ?

12. Contrôler les transactions annulées

```
SELECT
  statut_transaction,
  COUNT(*)              AS nombre_transactions,
  SUM(montant_net)      AS montant_net_total,
  SUM(montant_encaisse) AS montant_encaisse_total
FROM `VOTRE_PROJET.novatel.fact_transactions`
GROUP BY statut_transaction;
```

💬 Questions de réflexion

31. Repérez-vous des transactions annulées ?
32. Une transaction annulée doit-elle contribuer au chiffre d'affaires ?
33. Que devraient valoir son montant net et son montant encaissé ?

La règle métier retenue pour la vue propre sera :

```
CASE
  WHEN statut_transaction = 'Annulée' THEN 0
  ELSE montant_net
END
```

13. Rechercher les montants aberrants

```
SELECT
  MIN(montant_brut) AS minimum,
  AVG(montant_brut) AS moyenne,
  MAX(montant_brut) AS maximum
```

```
FROM `VOTRE_PROJET.novatel.fact_transactions`;
```

```
SELECT
  transaction_id,
  produit_id,
  montant_brut,
  montant_net
FROM `VOTRE_PROJET.novatel.fact_transactions`
ORDER BY montant_brut DESC
LIMIT 50;
```

💬 Questions de réflexion

34. Certaines valeurs vous paraissent-elles surprenantes ?
35. Un montant élevé est-il nécessairement une anomalie ?
36. Pourquoi le contexte métier est-il indispensable avant de corriger une valeur aberrante ?
37. Comment dim_produit peut-elle vous aider à juger ces montants ?

14. Vérifier la cohérence avec le catalogue produit

```
SELECT
  f.transaction_id,
  p.nom_produit,
  p.categorie_produit,
  p.prix_catalogue,
  f.montant_brut,
  f.remise,
  f.montant_net
FROM `VOTRE_PROJET.novatel.fact_transactions` AS f
LEFT JOIN `VOTRE_PROJET.novatel.dim_produit` AS p
  ON f.produit_id = p.produit_id
ORDER BY f.montant_brut DESC
LIMIT 50;
```

💬 Questions de réflexion

38. Certains montants sont-ils très éloignés du prix normal du produit ?
39. Comment distinguer remise importante, anomalie, remboursement, abonnement mensuel et vente de terminal ?

15. Contrôler les clés étrangères

Exemple avec les clients :

```
SELECT COUNT(*) AS clients_introuvables
FROM `VOTRE_PROJET.novatel.fact_transactions` AS f
LEFT JOIN `VOTRE_PROJET.novatel.dim_client` AS c
  ON f.client_id = c.client_id
WHERE c.client_id IS NULL;
```

Refaites le même contrôle avec produit, point de vente, date et type d'opération.

💬 **Questions de réflexion**

40. Trouvez-vous des références invalides ?
41. Pourquoi ce contrôle est-il important dans un modèle en étoile ?
42. Quel problème cela pourrait-il créer dans Looker Studio ?

16. Construire une vue propre compatible Sandbox

Le Sandbox ne permet pas les opérations DML classiques. Nous allons donc construire une vue de reporting qui applique les règles de nettoyage sans modifier la table brute.

📐 **Principe**

La table fact_transactions reste la source brute.
La vue vw_transactions_clean devient la source préparée pour le reporting.

La requête ci-dessous :

- écarte les lignes sans date ;
- neutralise les transactions annulées ;
- conserve une seule occurrence par transaction_id ;
- préserve la table brute.

```
CREATE OR REPLACE VIEW `VOTRE_PROJET.novatel.vw_transactions_clean` AS
SELECT * EXCEPT(rn)
FROM (
  SELECT
    transaction_id,
    date_id,
    client_id,
    produit_id,
    point_vente_id,
    type_operation_id,
    quantite,
    montant_brut,
    remise,
    remboursement,
    CASE
      WHEN statut_transaction = 'Annulée' THEN 0
      ELSE montant_net
    END AS montant_net,
    CASE
      WHEN statut_transaction = 'Annulée' THEN 0
      ELSE montant_encaisse
    END AS montant_encaisse,
    statut_transaction,
    ROW_NUMBER() OVER (
      PARTITION BY transaction_id
      ORDER BY transaction_id
    ) AS rn
  FROM `VOTRE_PROJET.novatel.fact_transactions`
  WHERE date_id IS NOT NULL
)
WHERE rn = 1;
```

⚠️ **Pourquoi une fonction ROW_NUMBER ici ?**

Elle sert uniquement à conserver une occurrence par transaction_id. Vous n'avez pas à apprendre cette syntaxe par cœur à ce stade : la requête vous est fournie pour appliquer proprement la règle de déduplication.

💬 **Questions de réflexion**

43. Pourquoi gardons-nous fact_transactions intacte ?
44. Pourquoi une vue est-elle adaptée à ce brief ?
45. Quelle anomalie est traitée par le WHERE ?
46. Quelle règle métier est appliquée avec CASE WHEN ?
47. Quelle règle de qualité est appliquée avec ROW_NUMBER ?

17. Vérifier la vue finale

```
SELECT COUNT(*) AS nombre_lignes
FROM `VOTRE_PROJET.novatel.vw_transactions_clean`;
```

```
SELECT
  SUM(montant_net)      AS ca_net,
  SUM(montant_encaisse) AS ca_encaisse
FROM `VOTRE_PROJET.novatel.vw_transactions_clean`;
```

💬 **Questions de réflexion**

48. Pourquoi le nombre de lignes est-il différent de la table brute ?
49. Pourquoi le CA peut-il avoir changé ?
50. Le CA encaissé est-il inférieur ou égal au CA net ?
51. Le résultat vous paraît-il plus cohérent ?

18. Produire quelques indicateurs de contrôle

Ces requêtes ne constituent pas encore le travail de datavisualisation. Elles servent à valider que les données préparées produisent des résultats cohérents avant le passage à Looker Studio.

18.1 CA net par région

```
SELECT
  p.region,
  SUM(f.montant_net) AS ca_net
FROM `VOTRE_PROJET.novatel.vw_transactions_clean` AS f
LEFT JOIN `VOTRE_PROJET.novatel.dim_point_vente` AS p
  ON f.point_vente_id = p.point_vente_id
GROUP BY p.region
ORDER BY ca_net DESC;
```

18.2 CA par catégorie de produit

```
SELECT
```

```
  p.categorie_produit,
  SUM(f.montant_net) AS ca_net
FROM `VOTRE_PROJET.novatel.vw_transactions_clean` AS f
LEFT JOIN `VOTRE_PROJET.novatel.dim_produit` AS p
  ON f.produit_id = p.produit_id
GROUP BY p.categorie_produit
ORDER BY ca_net DESC;
```

18.3 Nombre de transactions par type d'opération

```
SELECT
  o.type_operation,
  COUNT(*) AS nombre_transactions
FROM `VOTRE_PROJET.novatel.vw_transactions_clean` AS f
LEFT JOIN `VOTRE_PROJET.novatel.dim_type_operation` AS o
  ON f.type_operation_id = o.type_operation_id
GROUP BY o.type_operation
ORDER BY nombre_transactions DESC;
```

19. Questions finales de synthèse

💬 Questions de réflexion

52. Quelles étaient les principales anomalies détectées ?
53. Lesquelles pouvaient avoir un impact direct sur le chiffre d'affaires ?
54. Quelles anomalies avez-vous décidé d'exclure ou de corriger ?
55. Pourquoi avez-vous conservé la table brute ?
56. Quel est l'intérêt d'une vue BigQuery dans ce projet ?
57. Quelle différence faites-vous entre table brute et vue propre ?
58. Quelles données considérez-vous maintenant prêtes pour le reporting ?
59. Quels contrôles effectueriez-vous avant de connecter Looker Studio ?

20. Ressources officielles

- BigQuery Sandbox : https://cloud.google.com/bigquery/docs/sandbox
- Charger des données locales dans BigQuery : https://cloud.google.com/bigquery/docs/batch-loading-data
- BigQuery — démarrage rapide console : https://cloud.google.com/bigquery/docs/quickstarts/load-data-console

🎯 À retenir

Un dashboard fiable commence par des données fiables.
Dans le prochain brief, la vue propre créée ici sera utilisée comme source pour construire le reporting NovaTel dans Looker Studio.